

SDACS FIRMWARE ARCHITECTURE / README

PROJECT OVERVIEW

Smart Distributed Acoustic Calibration System (SDACS) is a multi-node embedded sensing platform for room-acoustic data acquisition. Each ESP32-S3 node acquires acoustic and environmental data, computes local features, and publishes structured telemetry to a Raspberry Pi 5 backend for visualization, logging, and future analysis.

FIRMWARE STRUCTURE AND TASKS

- **app_main**: initializes firmware configuration, NVS/settings, GPIO/peripheral setup, Wi-Fi/MQTT services, and application tasks.
- **Audio task**: samples the I2S MEMS microphone, calculates RMS/dBFS, performs FFT-based feature extraction, and prepares audio metrics.
- **MQTT/Wi-Fi task**: maintains broker connection, publishes telemetry, handles reconnect behavior, and reports communication status.
- **Sensor task**: reads I2C environmental sensors, formats readings, and publishes temperature/humidity telemetry.
- **Logging/config**: supports serial diagnostic output, SD-card concepts, and text-file based network/device configuration.

DATA FLOW

- Node boots and loads Wi-Fi, MQTT, and node identity configuration.
- I2S audio and I2C sensors are sampled on-device.
- Firmware computes compact features such as RMS, dBFS, FFT peak, and environmental readings.
- JSON payloads are published over MQTT through Wi-Fi.
- Raspberry Pi 5 runs Mosquitto/Node-RED for dashboarding, CSV/database logging, and system monitoring.

KEY INTERFACES AND TOPICS

Area	Implementation Notes
MCU / Framework	ESP32-S3 firmware developed in C using ESP-IDF; task-oriented structure with serial monitoring and Git-based revisions.
Acoustic Input	I2S MEMS microphone acquisition for room-acoustic measurements; features include RMS, dBFS, and FFT peak frequency.
Sensor Bus	I2C environmental sensor integration; address scanning and wiring verification used during troubleshooting.
MQTT Topics	sdacs/node/node01/features, sdacs/node/node01/features/audio, sdacs/node/node01/features/temp_humidity, and sdacs/node/node01/features/# for catch-all monitoring.
Backend	Raspberry Pi 5 running Mosquitto and Node-RED dashboards for real-time visualization, debugging, and storage workflows.

REPRESENTATIVE HARDWARE PINNING

One tested ESP32-S3 microphone mapping used BCLK=GPIO6, WS/LRCLK=GPIO5, DOUT=GPIO4, with microphone SEL tied to GND for left-channel operation. Later board mappings were adjusted per available GPIOs, reinforcing the need for clear pin documentation and hardware/software alignment.

DEBUGGING AND VALIDATION NOTES

- Used serial logs, I2C scans, MQTT broker monitoring, Node-RED debug panels, and GPIO/pin checks to isolate firmware and hardware issues.
- Diagnosed issues involving Wi-Fi reconnect behavior, dropped MQTT publish events, serial port conflicts, I2S pin naming confusion, and sensor bus visibility.
- Calibration workflow used SPL reference equipment and repeatable RMS/dBFS readings to estimate offsets and support validation documentation.
- Design emphasis: keep node firmware modular, publish compact structured telemetry, preserve useful diagnostics, and verify behavior through repeatable tests.

VALUE FOR EMBEDDED SOFTWARE ROLES

This project demonstrates practical embedded C development, hardware/software integration, communication-protocol troubleshooting, Linux/Raspberry Pi backend workflows, test documentation, and system-level thinking across a distributed sensing platform.

Portfolio note: *This summary is intended as a non-confidential overview of SDACS firmware architecture, testing workflow, and system integration experience for job applications.*